

JEGYZŐKÖNYV

Szakmai Gyakorlat II.

Frontend fejlesztési szakasz

Készítette: Orosz Kristóf

Neptunkód: EYZWG9

Dátum: 2026. június 23.



Orosz Kristóf

Hallgató

 **simpleSoft Kft.**
3525 Miskolc, Szentpáli u. 1.
Adószám: 13994761-2-05
-4-

Dédesi Péter

Szakmai konzulens

Tartalomjegyzék

Bevezetés	3
Fájlok tartalmának leírása	3
Menürendszer kialakítása	4
Weboldalak kinézetének megtervezése	10
Profil menüpont elkészítése	14
Regisztráció és bejelentkezés	19
Ingatlan funkciók kialakítása	27
Bérlő funkciók kialakítása	40
Szerződés funkciók kialakítása	47
Fizetés funkciók kialakítása	62
Üzenetkezelés kialakítása	72

Bevezetés

Az ingatlankezelő rendszer fejlesztésének frontend szakasza során a projekt kliensoldali komponenseinek kialakítása valósult meg. A fejlesztés célja egy modern, átlátható és felhasználóbarát felület létrehozása volt, amely lehetővé teszi az ingatlanok, bérlok, szerződések, fizetések és üzenetek kezelését. A frontend alkalmazás React keretrendszer használatával készült, amely támogatja a komponensalapú fejlesztést, valamint a dinamikus és hatékony felhasználói felületek kialakítását.

A fejlesztési folyamat során elkészítettem a rendszer navigációs menüjét, megterveztem és kialakítottam az egyes weboldalak felépítését, valamint megvalósítottam a felhasználói hitelesítéshez kapcsolódó regisztrációs és bejelentkezési felületeket.

Fájlok tartalmának leírása

- Bevezetés.pdf	[A frontend fejlesztési szakasz bevezetése.]
- Menürendszer kialakítása.pdf	[A navigációs menürendszer kialakítása.]
- Weboldalak kinézetének megtervezése.pdf	[A weboldalak felépítésének megtervezése.]
- Profil menüpont elkészítése.pdf	[A felhasználói profil oldal kialakítása.]
- Regisztráció és bejelentkezés.pdf	[A felhasználók felületeinek dokumentációja.]
- Ingatlan funkciók kialakítása.pdf	[Az ingatlanok kezelési funkciók.]
- Bérlok oldalak és funkciók beállítása.pdf	[A bérlok kezelési funkciók.]
- Szerződések funkciók beállítása.pdf	[A szerződések kezelési funkciók.]
- Fizetések létrehozása.pdf	[A fizetések kezelésének dokumentációja.]
- Üzenetek funkciók létrehozása és beállítása.pdf	[Az üzenetküldési funkciók.]
- Frontend forrásfájlok	[A frontend alkalmazás React komponensei.]

Menürendszer kialakítása

SORSZÁM	OLDAL	ÚTVONAL	FUNKCIÓ
1	Főoldal	/	A rendszer kezdőoldala.
2	Bejelentkezés Regisztráció	/bejelentkezés /regisztracio	Felhasználók hitelesítése és regisztráció.
3	Ingatlanok	/ingatlanok	Ingatlanok listázása és kezelése.
4	Bérlők	/berlok	Bérlők kezelése.
5	Szerződések	/szerzodesek	Szerződések kezelése.
6	Fizetések	/fizetesek	Fizetések kezelése.
7	Profil Üzenetek	/profil /uzenetek	Felhasználói adatok és kapcsolat.

A munkafolyamat során létrehoztam az alkalmazás alapvető szerkezetét, valamint kialakítottam a rendszer navigációs menüjét egy külön Menu komponens segítségével. Elkészítettem az App.js fájlt, amely a frontend alkalmazás központi komponenseként szolgál, valamint összekapcsoltam a menürendszert a főoldallal. A rendszer megjelenésének kialakításához létrehoztam az App.css stíluslapot, amelyben meghatároztam a fekete-sárga színvilágot, a menüsáv megjelenését és az alapvető felületi elemek formázását. A fejlesztés során törekedtem az egységes, átlátható és felhasználóbarát megjelenés kialakítására. Az elkészült frontend alapok megfelelő kiindulási pontot biztosítanak a későbbi oldalak, funkciók és backend kapcsolatok megvalósításához.

Menu.jsx

A Menu.jsx fájl elkészítése során kialakítottam az alkalmazás navigációs menürendszerét. A komponens feladata, hogy a felhasználók számára elérhetővé tegye a rendszer különböző oldalait egy központi menüsávon keresztül. A menü tartalmazza a főoldal, az ingatlanok, a bérlők, a szerződések, a fizetések és az üzenetek eléréséhez szükséges hivatkozásokat. A bal oldalon elhelyezett „Ingatlankezelő rendszer” felirat egy kattintható logóként működik, amely az ingatlanok oldalra navigálja a felhasználót. A komponens kialakítása során törekedtem az egyszerű, átlátható és felhasználóbarát navigáció megvalósítására, amely illeszkedik a rendszer fekete-sárga színvilágához.

```
import { Link, useNavigate } from "react-router-dom";

function Menu() {

  const navigate = useNavigate();

  const user =
    JSON.parse(localStorage.getItem("user"));

  const kijelentkezés = () => {

    localStorage.removeItem("user");

    navigate("/bejelentkezés");

  };

  return (
```

```
<header>

  <Link to="/" className="logo">
    Ingatlankezelő rendszer
  </Link>

  <nav>

    <Link to="/ingatlanok">
      Ingatlanok
    </Link>

    <Link to="/berlok">
      Bérlok
    </Link>

    <Link to="/szerzodesek">
      Szerződések
    </Link>

    <Link to="/fizetesek">
      Fizetések
    </Link>

    <Link to="/profil">
      Profil
    </Link>

    {!user && (

      <Link to="/bejelentkezes">
        Bejelentkezés
      </Link>

    )}

    {user && (

      <button
        onClick={kijelentkezes}
        className="logout-btn"
      >
        Kijelentkezés
      </button>

    )}

  </nav>
```

```
        </header>

    );
}

export default Menu;
```

App.css

Az App.css fájl elkészítése során kialakítottam az Ingatlankezelő rendszer alapvető megjelenését és stílusát. A stíluslap feladata a weboldal egységes arculatának biztosítása, valamint a különböző felületi elemek formázása. Meghatároztam a rendszer fekete-sárga színvilágát, amely modern és könnyen áttekinthető megjelenést biztosít a felhasználók számára. A CSS fájl tartalmazza a navigációs menü, a logó, a hivatkozások és az oldal alapvető elemeinek formázását is. Külön figyelmet fordítottam arra, hogy az interaktív elemek, például a menüpontok, vizuális visszajelzést adjanak az egérmutató ráhelyezésekor. A kialakított stílusok hozzájárulnak a rendszer felhasználóbarát és egységes megjelenésének megvalósításához.

```
body {
    margin: 0;
    font-family: Arial, sans-serif;
    background-color: #f5f5f5;
}

header {
    background-color: black;
    padding: 12px 20px;

    display: flex;
    align-items: center;
}

.logo {
    color: yellow;
    text-decoration: none;
    font-size: 28px;
    font-weight: bold;
}

nav {
    margin-left: auto; /* jobbra tolja a menüt */
    display: flex;
    gap: 30px;
}

nav a {
    color: yellow;
```

```
    text-decoration: none;
    font-size: 18px;
    font-weight: bold;
}

nav a:hover {
    color: white;
}

h1 {
    text-align: center;
    margin-top: 50px;
}

p {
    text-align: center;
}

.logout-btn {

    background: none;
    border: none;

    color: yellow;

    font-size: 18px;
    font-weight: bold;

    cursor: pointer;
}

.logout-btn:hover {

    color: white;
}
```

App.js

Az App.js fájl elkészítése során kialakítottam a React alkalmazás központi komponensét, amely a rendszer fő belépési pontjaként működik. A fejlesztés során összekapcsoltam a korábban elkészített Menu komponenst az alkalmazással, így a navigációs menü minden oldalon megjelenhet. Az oldalon elhelyeztem az Ingatlankezelő rendszer címét, valamint egy rövid üdvözlő szöveget, amely tájékoztatja a felhasználót az alkalmazás céljáról.

```
import './App.css';

import { BrowserRouter, Routes, Route } from 'react-router-dom';
```

```
import Menu from "../components/Menu";
import Bejelentkezes from "../components/Bejelentkezes";
import Regisztracio from "../components/Regisztracio";

function Fooldal() {
  return <h1>Főoldal</h1>;
}

function Ingatlanok() {
  return <h1>Ingatlanok oldal</h1>;
}

function Berlok() {
  return <h1>Bérlők oldal</h1>;
}

function Szerzodesek() {
  return <h1>Szerződések oldal</h1>;
}

function Fizetesek() {
  return <h1>Fizetések oldal</h1>;
}

function Profil() {
  return (
    <div>

      <h1>Profil oldal</h1>

      <p>
        Itt jelennek meg a felhasználó adatai.
      </p>

      <p>
        Innen érhető el az üzenetek kezelése.
      </p>

    </div>
  );
}

function Uzenetek() {
  return <h1>Üzenetek oldal</h1>;
}
```



```
}  
  
function App() {  
  
  return (  
  
    <BrowserRouter>  
  
      <Menu />  
  
      <Routes>  
  
        <Route  
          path="/"  
          element={<Fooldal />}  
        />  
  
        <Route  
          path="/ingatlanok"  
          element={<Ingatlanok />}  
        />  
  
        <Route  
          path="/berlok"  
          element={<Berlok />}  
        />  
  
        <Route  
          path="/szerzodesek"  
          element={<Szerzodesek />}  
        />  
  
        <Route  
          path="/fizetesek"  
          element={<Fizetesek />}  
        />  
  
        <Route  
          path="/profil"  
          element={<Profil />}  
        />  
  
        <Route  
          path="/uzenetek"  
          element={<Uzenetek />}  
        />  
  
        <Route
```

```
        path="/bejelentkezes"
        element={<Bejelentkezes />}
      />

      <Route
        path="/regisztracio"
        element={<Regisztracio />}
      />

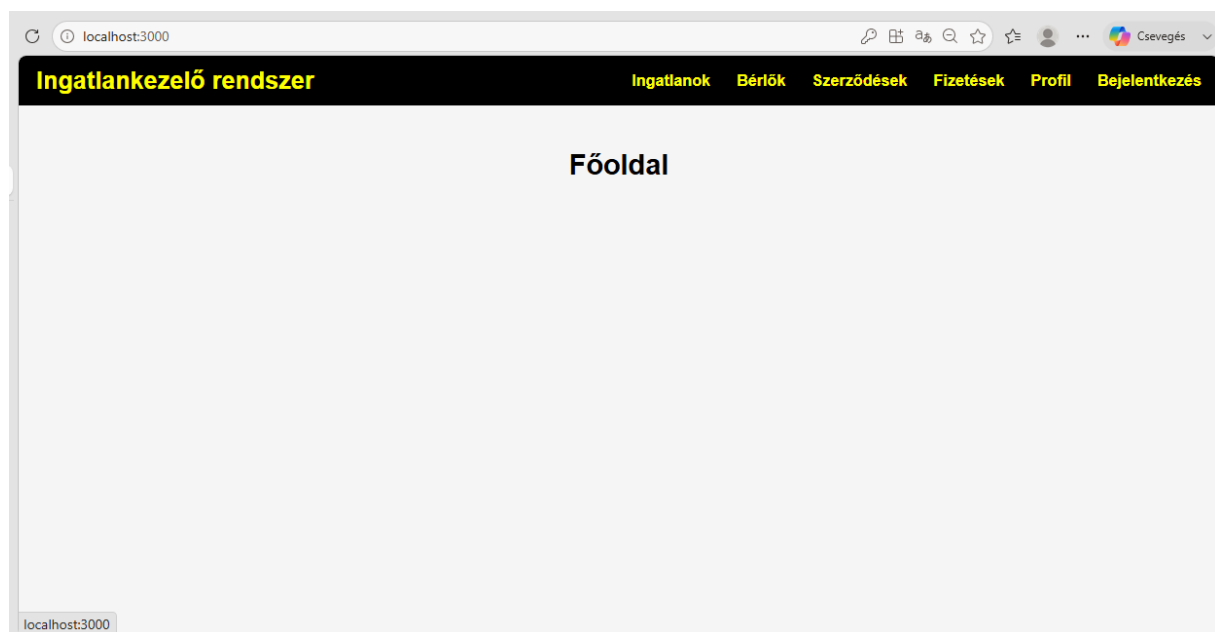
    </Routes>

  </BrowserRouter>

);
}

export default App;
```

Menüsáv kimenete:



1. kép: Ingatlankezelő rendszer menüsávja.

Weboldalak kinézetének megtervezése

HTML oldalak és azok funkcióinak megtervezése.

SORSZÁM	HTML OLDAL	FUNKCIÓK	TÍPUS	JOGOSULTSÁG
1.	index.html, felhasznalo.html, admin.html	Főoldal	HTML	Mindkettő
2.	bejelentkezes.html	Bejelentkezési oldal	HTML	Mindkettő

3.	regisztracio.html	Regisztrációs oldal	HTML	Mindkettő
4.	ingatlanok.html	Ingatlanok listázása	HTML	Mindkettő
		Ingatlan létrehozása	Funkció	Admin
		Ingatlan módosítása	Funkció	Admin
		Ingatlan törlése	Funkció	Admin
5.	berlok.html	Bérlők listázása	HTML	Admin
		Bérlő létrehozása	Funkció	Admin
		Bérlő módosítása	Funkció	Admin
		Bérlő törlése	Funkció	Admin
		Bérlő hozzárendelése felhasználóhoz	Funkció	Admin
6.	szerzodesek.html	Szerződések listázása	HTML	Admin
		Szerződés létrehozása	Funkció	Admin
		Szerződés módosítása	Funkció	Admin
		Szerződés törlése	Funkció	Admin
7.	fizetesek.html	Fizetések listázása	HTML	Admin
		Fizetés létrehozása	Funkció	Admin
		Fizetés módosítása	Funkció	Admin
		Fizetés törlése	Funkció	Admin
8.	uzenetek.html	Üzenetek listázása	HTML	Admin
		Üzenet küldése	Funkció	Felhasználó
9.	fiokom.html	Saját adatok megtekintése	HTML	Felhasználó
		Saját szerződések megtekintése	Funkció	Felhasználó
		Saját fizetések megtekintése	Funkció	Felhasználó
		Saját üzenetek megtekintése	Funkció	Felhasználó

1. Általános stílusirányelv

- **Színvilág:** fekete és sárga színekombináció.
- **Háttér:** fekete színű.
- **Feliratok:** fehér színűek a jó olvashatóság érdekében.
- **Kiemelések és fontos elemek:** sárga színnel jelennek meg.
- **Betűtípus:** alapértelmezett.
- **Gombok stílusa:** lekerekített szegély, fehér színű feliratokkal.

2. Főoldal (index.html, felhasznalo.html, admin.html)

- Tartalom: bemutatkozó szöveg és 2-3 információs szekció.
- Gombok:
 - „Bejelentkezés”
 - „Regisztráció”
 - „Ingatlanok megtekintése”
- A gombok sárga színűek, lekerekített kialakítással.
- PHP/Backend kapcsolat: nincs.

3. Bejelentkezés (bejelentkezés.html)

- Űrlap:
 - E-mail
 - Jelszó
- „Bejelentkezés” gomb.
- Gomb, amely a regisztrációs oldalra mutat.
- Sikeres bejelentkezés után a felhasználó szerepkörének megfelelő oldal jelenik meg.
- Backend kapcsolat:
 - /api/bejelentkezés

4. Regisztráció (regisztracio.html)

- Űrlap:
 - Név
 - E-mail
 - Jelszó
 - Telefonszám
 - Szerepkör
- „Regisztráció” gomb.
- Backend kapcsolat:
 - /api/regisztracio

5. Ingatlanok (ingatlanok.html)

- Kártyás megjelenítés.
- Megjelenő adatok:
 - Cím
 - Típus
 - Szobaszám
 - Havi bérleti díj
 - Leírás

Admin funkciók

- Új ingatlan létrehozása
- Ingatlan módosítása
- Ingatlan törlése

Backend kapcsolatok

- /api/ingatlanlistazas
- /api/ingatlanletrehozas
- /api/ingatlanmodositas
- /api/inglantorles

6. Bérlok (berlok.html)

- Táblázatos megjelenítés.
- Megjelenő adatok:
 - Név
 - E-mail
 - Telefonszám
 - Lakcím

Admin funkciók

- Bérlok létrehozása

- Bérleti módosítása
- Bérleti törlése
- Bérleti hozzárendelése felhasználóhoz

Backend kapcsolatok

- /api/berlorehozas
- /api/berloistazas
- /api/berlomodositas
- /api/berlotorles
- /api/berlohozzarendeles

7. Szerződések (szerzodesek.html)

- Táblázatos megjelenítés.
- Megjelenő adatok:
 - Ingatlan
 - Bérleti
 - Kezdő dátum
 - Végdátum
 - Havi díj
 - Státusz

Admin funkciók

- Új szerződés létrehozása
- Szerződés módosítása
- Szerződés törlése

Backend kapcsolatok

- /api/szerzodesletrehozas
- /api/szerzodeslistazas
- /api/szerzodesmodositas
- /api/szerzodestorles

8. Fizetések (fizetesek.html)

- Táblázatos megjelenítés.
- Megjelenő adatok:
 - Szerződés
 - Összeg
 - Fizetés dátuma
 - Fizetési mód
 - Státusz

Admin funkciók

- Fizetés rögzítése
- Fizetés módosítása
- Fizetés törlése

Backend kapcsolatok

- /api/fizetesletrehozas
- /api/fizeteslistazas
- /api/fizetesmodositas
- /api/fizetestorles

Profil menüpont elkészítése

Profil.jsx

A fejlesztés során elkészítettem a Profil menüpontot, amely a bejelentkezett felhasználó adatainak megjelenítésére szolgál. A profil oldal külön React komponensként került létrehozásra. A komponens a localStorage-ban tárolt felhasználói adatokat olvassa ki, amelyek a sikeres bejelentkezést követően kerülnek eltárolásra. Ennek segítségével az aktuálisan bejelentkezett felhasználó adatai jelennek meg a felületen.

```
import "../styles/profil.css";
import { Link } from "react-router-dom";

function Profil() {

  const user = JSON.parse(
    localStorage.getItem("user")
  );

  if (!user) {

    return (

      <div className="profil-container">

        <div className="profil-card">

          <h1>Profil</h1>

          <p>
            Nincs bejelentkezett felhasználó.
          </p>

        </div>

      </div>

    );

  }

  return (

    <div className="profil-container">

      <div className="profil-card">

        <h1>Profilom</h1>
```

```
<div className="adat-sor">

    <span>Név:</span>

    <strong>
        {user.nev || "-"}
    </strong>

</div>

<div className="adat-sor">

    <span>E-mail:</span>

    <strong>
        {user.email || "-"}
    </strong>

</div>

<div className="adat-sor">

    <span>Telefonszám:</span>

    <strong>
        {user.telefonszam || "-"}
    </strong>

</div>

<div className="adat-sor">

    <span>Személyi szám:</span>

    <strong>
        {user.szemelyiszam || "-"}
    </strong>

</div>

<div className="adat-sor">

    <span>Lakcím:</span>

    <strong>
        {user.lakcim || "-"}
    </strong>

</div>
```

```
        </div>

        <div className="adat-sor">

            <span>Jelszó:</span>

            <strong>
                {user.jelszo || "-"}
            </strong>

        </div>

        <div className="adat-sor">

            <span>Szerepkör:</span>

            <strong>
                {user.szerepkor || "-"}
            </strong>

        </div>

        <div className="profil-menu">

            <Link
                to="/uzenetek"
                className="profil-gomb"
            >
                Üzeneteim
            </Link>

        </div>

    </div>

</div>

);

}

export default Profil;
```

profil.css

A megjelenés kialakításához külön CSS állományt készítettem. A stíluslap segítségével kártyás megjelenítést, árnyékokat, kiemelt adatmezőket és egységes színvilágot alkalmaztam, amely illeszkedik az alkalmazás többi oldalának megjelenéséhez.


```
.profil-container {  
  
    display: flex;  
    justify-content: center;  
    align-items: center;  
  
    min-height: 80vh;  
  
}  
  
.profil-card {  
  
    width: 600px;  
  
    background: white;  
  
    border-radius: 12px;  
  
    padding: 40px;  
  
    box-shadow: 0 0 20px rgba(0,0,0,0.2);  
  
}  
  
.profil-card h1 {  
  
    text-align: center;  
  
    margin-bottom: 30px;  
  
}  
  
.adat-sor {  
  
    display: flex;  
  
    justify-content: space-between;  
  
    padding: 15px;  
  
    margin-bottom: 10px;  
  
    background-color: #f5f5f5;  
  
    border-left: 5px solid yellow;  
  
    border-radius: 5px;  
  
}
```

```
}

.profil-menu {

    margin-top: 30px;

    display: flex;

    justify-content: center;

}

.profil-gomb {

    background-color: black;

    color: yellow;

    text-decoration: none;

    padding: 12px 25px;

    border-radius: 5px;

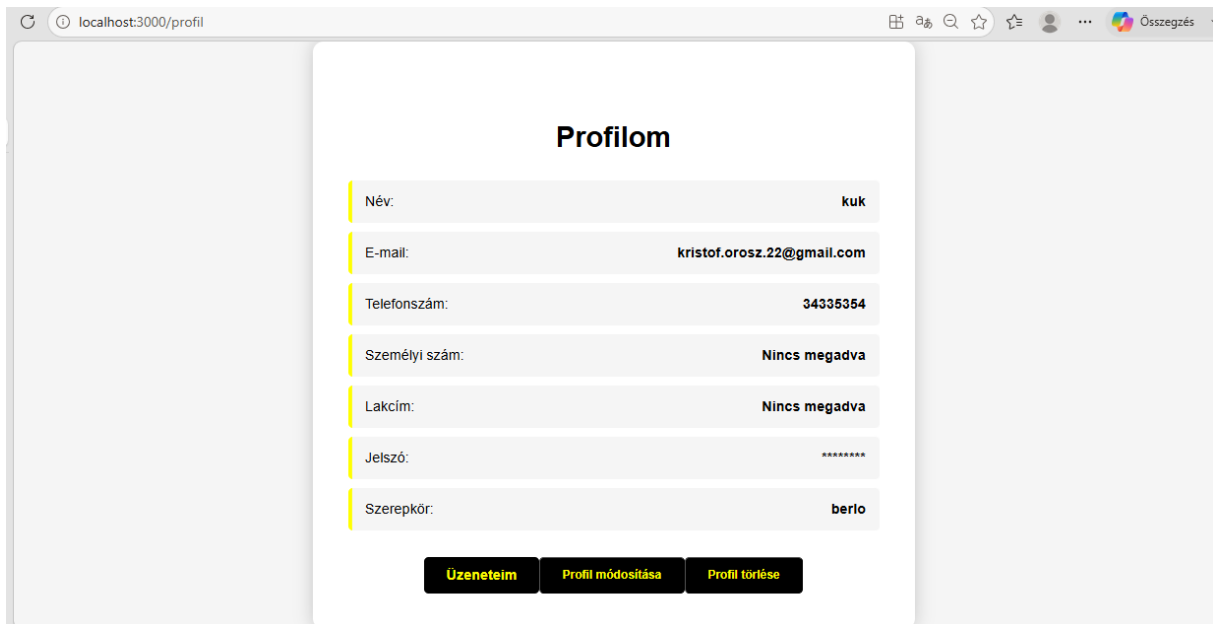
    font-weight: bold;

}

.profil-gomb:hover {

    color: white;

}
```



1.kép: Profil oldal HTML oldala

Regisztráció és bejelentkezés

Regisztráció

Regisztracio.jsx

A frontend fejlesztése során elkészítettem a rendszer regisztrációs oldalát React keretrendszer használatával. Az oldal lehetővé teszi új felhasználók számára a rendszerbe történő regisztrációt név, e-mail cím, jelszó, telefonszám és szerepkör megadásával. A fejlesztés során React useState hook segítségével kezeltem az űrlapmezők adatait, majd a megadott adatokat fetch API használatával továbbítottam a backend regisztrációs végpontja felé. A rendszer sikeres regisztráció esetén visszajelző üzenetet jelenít meg, majd automatikusan átirányítja a felhasználót a bejelentkezési oldalra. A felhasználói felület kialakításához külön hitelesites.css stíluslapot alkalmaztam, amely biztosítja az egységes és felhasználóbarát megjelenést. Az elkészült funkció működését a backend API-val összekapcsolva teszteltem, és sikeresen ellenőriztem az új felhasználók létrehozását.

```
import { Link, useNavigate } from "react-router-dom";
import { useState } from "react";
import "../styles/hitelesites.css";

function Regisztracio() {

  const [nev, setNev] = useState("");
  const [email, setEmail] = useState("");
  const [jelszo, setJelszo] = useState("");
  const [telefonszam, setTelefonszam] = useState("");
  const [szerepkor, setSzerepkor] = useState("felhasznalo");

  const navigate = useNavigate();

  const regisztracio = async (e) => {
```

```
e.preventDefault();

try {

  const valasz = await fetch(
    "http://localhost:3001/api/regisztracio",
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({
        nev,
        email,
        jelszo,
        szerepkor,
        telefonszam
      })
    }
  );

  const adat = await valasz.json();

  alert(adat.uzenet);

  if (adat.siker) {

    navigate("/bejelentkezés");

  }

} catch (err) {

  console.error(err);

  alert("Hiba történt!");

}

};

return (

  <div className="auth-container">

    <div className="auth-card">

      <h2>Regisztráció</h2>
```

```
<form onSubmit={regisztracio}>

  <input
    type="text"
    placeholder="Név"
    value={nev}
    onChange={(e) => setNev(e.target.value)}
  />

  <input
    type="email"
    placeholder="E-mail"
    value={email}
    onChange={(e) => setEmail(e.target.value)}
  />

  <input
    type="password"
    placeholder="Jelszó"
    value={jelszo}
    onChange={(e) => setJelszo(e.target.value)}
  />

  <input
    type="text"
    placeholder="Telefonszám"
    value={telefonszam}
    onChange={(e) => setTelefonszam(e.target.value)}
  />

  <select
    value={szerepkor}
    onChange={(e) => setSzerepkor(e.target.value)}
  >
    <option value="felhasznalo">
      Felhasználó
    </option>

    <option value="admin">
      Admin
    </option>
  </select>

  <button type="submit">
    Regisztráció
  </button>
```

```
    </form>

    <p>

      Már rendelkezik fiókkal?{" "}

      <Link to="/bejelentkezes">
        Jelentkezzen be!
      </Link>

    </p>

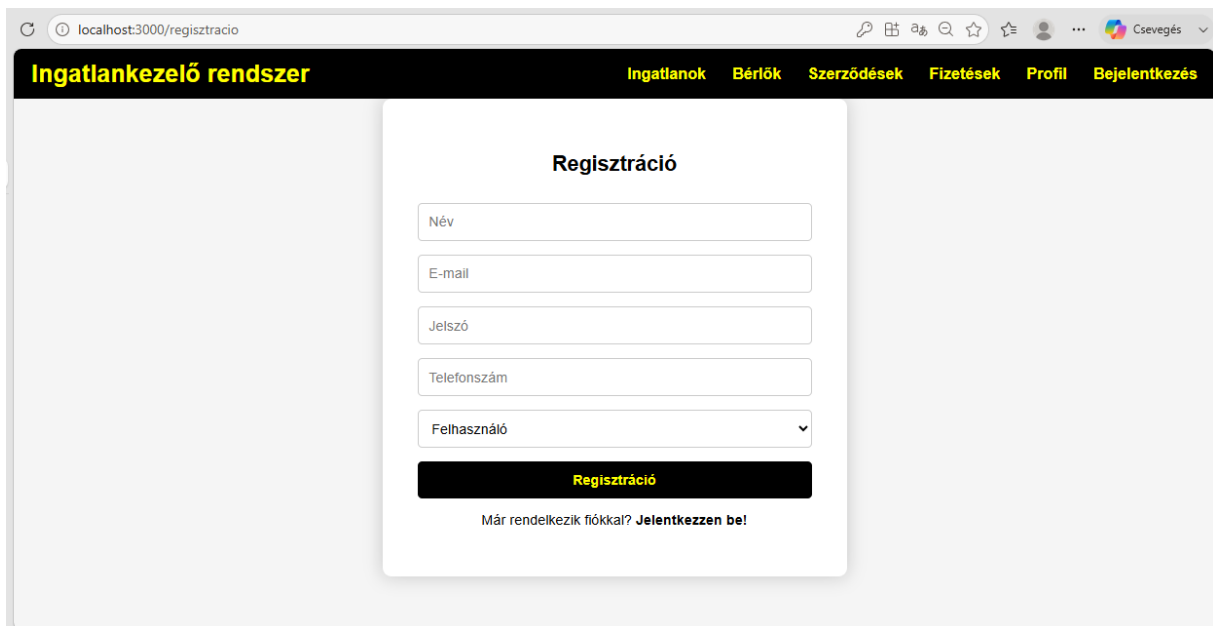
  </div>

</div>

);
}

export default Regisztracio;
```

1.Részlet a regisztracio.jsx fájlból.



2.Kép a regisztráció oldalról.

Bejelentkezes.jsx

A frontend fejlesztése során elkészítettem a rendszer bejelentkezési felületét React keretrendszer használatával. Az oldal lehetővé teszi a felhasználók számára az e-mail címük és jelszavuk megadásával történő hitelesítést. A fejlesztés során React useState hook segítségével kezeltem a felhasználó által megadott adatokat, majd a fetch API alkalmazásával továbbítottam azokat a backend bejelentkezési végpontja felé. Sikeres bejelentkezés esetén a rendszer eltárolja a felhasználó adatait a böngésző localStorage tárhelyén, majd automatikusan átirányítja az Ingatlanok oldalra. Hibás bejelentkezési

adatok esetén a felhasználó megfelelő hibaüzenetet kap a rendszerből. Az elkészült felület megjelenését a hitelesites.css stíluslap biztosítja, amely egységes és felhasználóbarát kialakítást nyújt a hitelesítési funkciók számára.

```
import { Link, useNavigate } from "react-router-dom";
import { useState } from "react";
import "../styles/hitelesites.css";

function Bejelentkezes() {

  const [email, setEmail] = useState("");
  const [jelszo, setJelszo] = useState("");

  const navigate = useNavigate();

  const bejelentkezes = async (e) => {

    e.preventDefault();

    try {

      const valasz = await fetch(
        "http://localhost:3001/api/bejelentkezes",
        {
          method: "POST",
          headers: {
            "Content-Type": "application/json"
          },
          body: JSON.stringify({
            email,
            jelszo
          })
        }
      );

      const adat = await valasz.json();

      if (adat.siker) {

        localStorage.setItem(
          "user",
          JSON.stringify(adat.user)
        );

        alert(adat.uzenet);

        navigate("/ingatlanok");

      } else {
```

```
        alert(adat.uzenet);

    }

    } catch (err) {

        console.error(err);

        alert("Hiba történt!");

    }

};

return (

    <div className="auth-container">

        <div className="auth-card">

            <h2>Bejelentkezés</h2>

            <form onSubmit={bejelentkezés}>

                <input
                    type="email"
                    placeholder="E-mail"
                    value={email}
                    onChange={(e) => setEmail(e.target.value)}
                />

                <input
                    type="password"
                    placeholder="Jelszó"
                    value={jelszo}
                    onChange={(e) => setJelszo(e.target.value)}
                />

                <button type="submit">
                    Bejelentkezés
                </button>

            </form>

            <p>

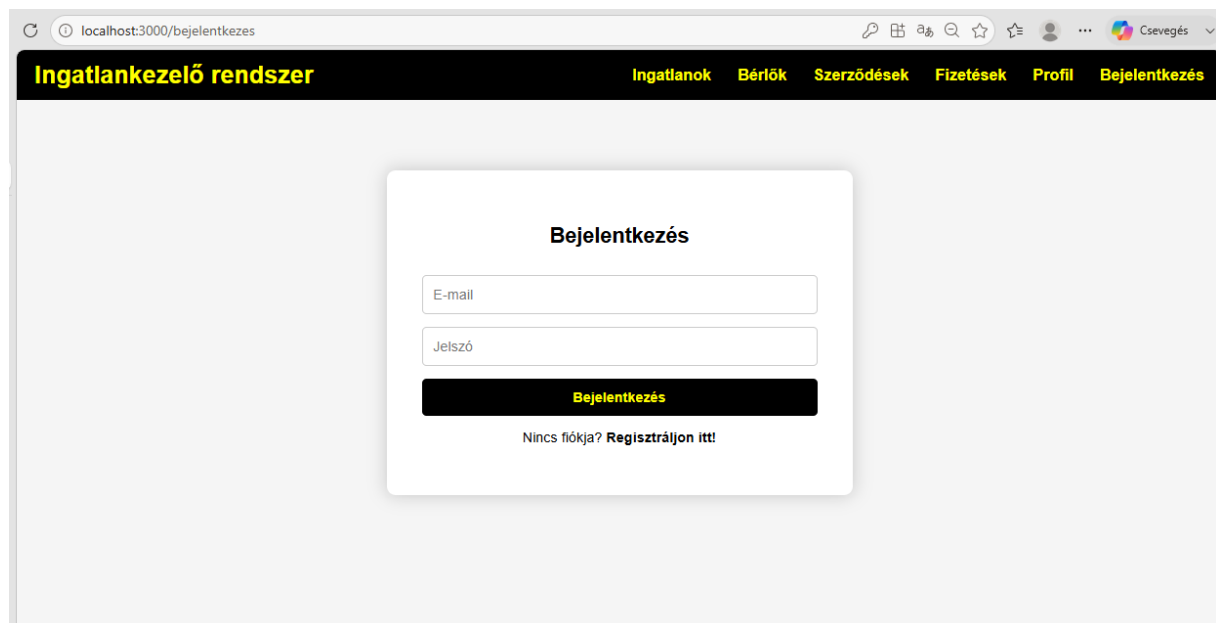
                Nincs fiókja? {" "}
```



```
        <Link to="/regisztracio">  
          Regisztráljon itt!  
        </Link>  
  
      </p>  
  
    </div>  
  
  </div>  
  
);  
}
```

```
export default Bejelentkezés;
```

3. Részlet a regisztracio.jsx fájlból.



4. Kép a bejelentkezés oldalról.

hitelesites.css

A frontend fejlesztése során elkészítettem a hitelesítési felületek megjelenését biztosító hitelesites.css stíluslapot. A stíluslap feladata a bejelentkezési és regisztrációs oldalak egységes, modern és felhasználóbarát megjelenésének kialakítása. A fejlesztés során középre igazítottam az űrlapokat, valamint egy kártya alapú elrendezést alkalmaztam, amely kiemeli a felhasználói adatbeviteli felületet. Meghatároztam a beviteli mezők, legördülő listák és gombok megjelenését, biztosítva azok könnyű használhatóságát és áttekinthetőségét. A rendszer színvilágához illeszkedve a gombok fekete háttérrel és sárga szöveggel jelennek meg, valamint egérmutató ráhelyezésekor vizuális visszajelzést adnak.

```
.auth-container {  
  
  display: flex;
```

```
    justify-content: center;
    align-items: center;

    min-height: 80vh;
}

.auth-card {

    background-color: white;

    width: 450px;

    padding: 40px;

    border-radius: 10px;

    box-shadow: 0 0 20px rgba(0, 0, 0, 0.2);
}

.auth-card h2 {

    text-align: center;
    margin-bottom: 30px;
}

.auth-card input,
.auth-card select {

    width: 100%;

    padding: 12px;

    margin-bottom: 15px;

    border: 1px solid #ccc;
    border-radius: 5px;

    font-size: 16px;

    box-sizing: border-box;
}

.auth-card button {
```

```
width: 100%;

background-color: black;
color: yellow;

border: none;

padding: 12px;

border-radius: 5px;

font-size: 16px;
font-weight: bold;

cursor: pointer;
}

.auth-card button:hover {

    background-color: #222;
}

.auth-card p {

    text-align: center;
}

.auth-card a {

    color: black;
    font-weight: bold;
    text-decoration: none;
}

.auth-card a:hover {

    text-decoration: underline;
}
```

5.Részlet a hitelesites.css fájlból.

Ingatlan funkciók kialakítása

Ingatlanok.jsx

Az Ingatlanok.jsx komponensben elkészítettem az ingatlanok kezelésére szolgáló felhasználói felületet. Megvalósítottam az ingatlanok listázását, amely a backendből tölti be az adatokat és kártyás formában jeleníti meg azokat. Létrehoztam az új ingatlan felvételéhez szükséges űrlapot, amely csak gombnyomásra jelenik meg a felhasználó számára. Kialakítottam az ingatlanok módosításának és törlésének lehetőségét, amelyek a megfelelő backend végpontokat hívják meg.

```
import { useEffect, useState } from "react";
import "../styles/ingatlanok.css";

function Ingatlanok() {

  const [ingatlanok, setIngatlanok] = useState([]);

  const [urlapLathato, setUrlapLathato] =
    useState(false);

  const [szerkesztettIngatlan,
    setSzerkesztettIngatlan] =
    useState(null);

  const [ujIngatlan, setUjIngatlan] = useState({
    tulajdonosid: "",
    cim: "",
    tipus: "",
    szobakszama: "",
    haviar: "",
    leiras: ""
  });

  const lekeres = async () => {

    try {

      const valasz = await fetch(
        "http://localhost:3001/api/ingatlanlistazas"
      );

      const adatok = await valasz.json();

      setIngatlanok(adatok);

    } catch (err) {

      console.error(err);

    }

  };

};
```

```
useEffect(() => {

    lekeres();

}, []);

const szerkesztes = (ingatlan) => {

    setSzerkesztettIngatlan(ingatlan);

    setUjIngatlan({
        tulajdonosid: ingatlan.tulajdonosid,
        cim: ingatlan.cim,
        tipus: ingatlan.tipus,
        szobakszama: ingatlan.szobakszama,
        haviar: ingatlan.haviar,
        leiras: ingatlan.leiras
    });

    setUrlapLathato(true);

};

const torles = async (id) => {

    if (!window.confirm("Biztosan törli az ingatlant?")) {
        return;
    }

    try {

        const valasz = await fetch(
            `http://localhost:3001/api/inglantorles/${id}`,
            {
                method: "DELETE"
            }
        );

        const adat = await valasz.json();

        alert(adat.uzenet);

        lekeres();

    } catch (err) {

        console.error(err);
    }
}
```

```
    }

    };

    const létrehozas = async (e) => {

        e.preventDefault();

        try {

            if (szerkesztettIngatlan) {

                const valasz = await fetch(
                    `http://localhost:3001/api/ingatlanmodositas/${szerkesztettIngatlan.id}`
                ,
                    {
                        method: "PUT",
                        headers: {
                            "Content-Type":
                                "application/json"
                        },
                        body: JSON.stringify(
                            ujIngatlan
                        )
                    }
                );

                const adat =
                    await valasz.json();

                alert(adat.uzenet);

                setSzerkesztettIngatlan(null);

                setUrlapLathato(false);

                lekeres();

                return;
            }

            const valasz = await fetch(
                "http://localhost:3001/api/ingatlanletrehozas",
                {
                    method: "POST",
                    headers: {
                        "Content-Type":
                            "application/json"
                    }
                }
            );
```

```
        },
        body: JSON.stringify(
            ujIngatlan
        )
    }
});

const adat = await valasz.json();

alert(adat.uzenet);

setUjIngatlan({
    tulajdonosid: "",
    cim: "",
    tipus: "",
    szobakszama: "",
    haviar: "",
    leiras: ""
});

setUrlapLathato(false);

lekeres();

} catch (err) {

    console.error(err);

    alert("Hiba történt!");

}

};

return (

    <div className="ingatlan-container">

        <h1>Ingatlanok</h1>

        <button
            className="uj-btn"
            onClick={() =>
                setUrlapLathato(
                    !urlapLathato
                )
            }
        >
```

```
        {
          urlaplathato
            ? "Úrlap bezárása"
            : "Új ingatlan felvétele"
        }
      </button>

      {urlaplathato && (

        <form
          className="ingatlan-form"
          onSubmit={letrehozas}
        >

          <input
            type="number"
            placeholder="Tulajdonos ID"
            value={
              ujIngatlan.tulajdonosid
            }
            onChange={(e) =>
              setUjIngatlan({
                ...ujIngatlan,
                tulajdonosid:
                  e.target.value
              })
            }
          />

          <input
            type="text"
            placeholder="Cím"
            value={ujIngatlan.cim}
            onChange={(e) =>
              setUjIngatlan({
                ...ujIngatlan,
                cim: e.target.value
              })
            }
          />

          <input
            type="text"
            placeholder="Típus"
            value={ujIngatlan.tipus}
            onChange={(e) =>
              setUjIngatlan({
                ...ujIngatlan,
```



```
        tipus:
          e.target.value
        })
      }
    />

    <input
      type="number"
      placeholder="Szobák száma"
      value={
        ujIngatlan.szobakszama
      }
      onChange={(e) =>
        setUjIngatlan({
          ...ujIngatlan,
          szobakszama:
            e.target.value
        })
      }
    />

    <input
      type="number"
      placeholder="Havi ár"
      value={
        ujIngatlan.haviar
      }
      onChange={(e) =>
        setUjIngatlan({
          ...ujIngatlan,
          haviar:
            e.target.value
        })
      }
    />

    <textarea
      placeholder="Leírás"
      value={
        ujIngatlan.leiras
      }
      onChange={(e) =>
        setUjIngatlan({
          ...ujIngatlan,
          leiras:
            e.target.value
        })
      }
    />
```

```
        />

        <button type="submit">

            {
                szerkesztettIngatlan
                ? "Módosítás mentése"
                : "Új ingatlan felvétele"
            }

        </button>

    </form>

    )}

    <div className="ingatlan-grid">

        {ingatlanok.map((ingatlan) => (

            <div
                className="ingatlan-card"
                key={ingatlan.id}
            >

                <h3>
                    {ingatlan.cim}
                </h3>

                <p>
                    <strong>
                        Típus:
                    </strong>{" "}
                    {ingatlan.tipus}
                </p>

                <p>
                    <strong>
                        Szobák száma:
                    </strong>{" "}
                    {ingatlan.szobakszama}
                </p>

                <p>
                    <strong>
                        Havi ár:
                    </strong>{" "}
                    {ingatlan.haviar}
```

```
        {" "}Ft
    </p>

    <p>
        <strong>
            Leírás:
        </strong>{" "}
        {ingatlan.leiras}
    </p>

    <div className="gombok">

        <button
            className="modositas-btn"
            onClick={() =>
                szerkesztes(
                    ingatlan
                )
            }
        >
            Módosítás
        </button>

        <button
            className="torles-btn"
            onClick={() =>
                torles(
                    ingatlan.id
                )
            }
        >
            Törlés
        </button>

    </div>

</div>

    )})

</div>

</div>

);

}
```

```
export default Ingatlanok;
```

1. Részlet az *Ingatlan.jsx* fájlból.

ingatlanok.css

Az ingatlanok.css fájlban elkészítettem az Ingatlanok oldal teljes megjelenését és formázását. Kialakítottam a kártyás nézetet, amely áttekinthető módon jeleníti meg az ingatlanok adatait. Stílust adtam az űrlapoknak, beviteli mezőknek és gomboknak annak érdekében, hogy a felület felhasználóbarát legyen. A módosítás és törlés gombok külön színeket kaptak, így a funkciók könnyen megkülönböztethetők.

```
.ingatlan-container {
  max-width: 1300px;
  margin: 0 auto;
  padding: 20px;
}

.ingatlan-container h1 {
  text-align: center;
  margin-bottom: 15px;
}

.uj-btn {
  display: block;
  margin: 0 auto 15px auto;
  background-color: #007bff;
  color: white;
  border: none;
  padding: 10px 20px;
  border-radius: 6px;
  font-weight: bold;
  cursor: pointer;
}

.ingatlan-form {
  background: white;
  padding: 15px;
  border-radius: 8px;
  margin-bottom: 20px;
  box-shadow: 0 0 8px rgba(0,0,0,0.1);
}

.ingatlan-form input,
.ingatlan-form textarea {
  width: 100%;
  padding: 10px;
  margin-bottom: 8px;
  border: 1px solid #ccc;
  border-radius: 5px;
}
```

```
    box-sizing: border-box;
}

.ingatlan-form textarea {
    min-height: 70px;
    resize: vertical;
}

.ingatlan-form button {
    width: 100%;
    background: #28a745;
    color: white;
    border: none;
    padding: 10px;
    border-radius: 6px;
    cursor: pointer;
    font-weight: bold;
}

.ingatlan-grid {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(320px, 1fr));
    gap: 12px;
}

.ingatlan-card {
    background: white;
    border-radius: 8px;
    padding: 15px;
    box-shadow: 0 0 8px rgba(0,0,0,0.1);
}

.ingatlan-card h3 {
    margin: 0 0 10px 0;
    color: #007bff;
}

.ingatlan-card p {
    margin: 4px 0;
}

.gombok {
    display: flex;
    gap: 8px;
    margin-top: 12px;
}

.modositás-btn,
```

```
.torles-btn {  
    flex: 1;  
    border: none;  
    padding: 8px;  
    border-radius: 6px;  
    font-weight: bold;  
    cursor: pointer;  
}  
  
.modositas-btn {  
    background: orange;  
    color: white;  
}  
  
.torles-btn {  
    background: #dc3545;  
    color: white;  
}
```

2. Részlet az ingatlan.css fájlból.

localhost:3000/ingatlanok

Ingatlankezelő rendszer

Ingatlanok Bérlők Szerződések Fizetések Profil Kijelentkezés

Ingatlanok

Űrlap bezárása

Tulajdonos ID

Cím

Típus

Szobák száma

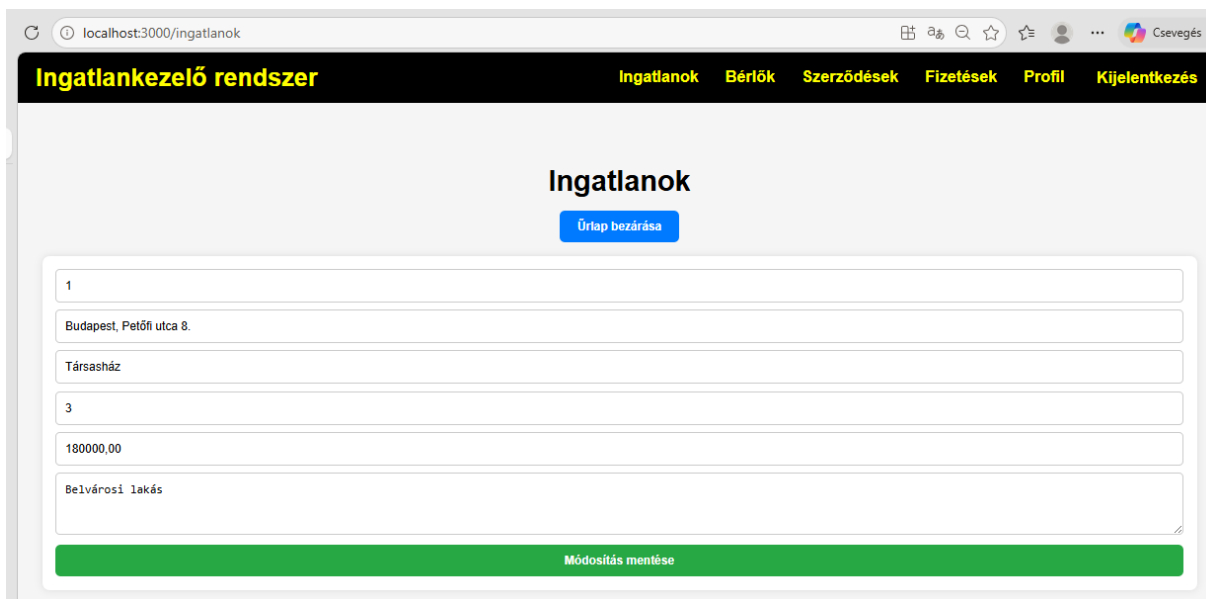
Havi ár

Leírás

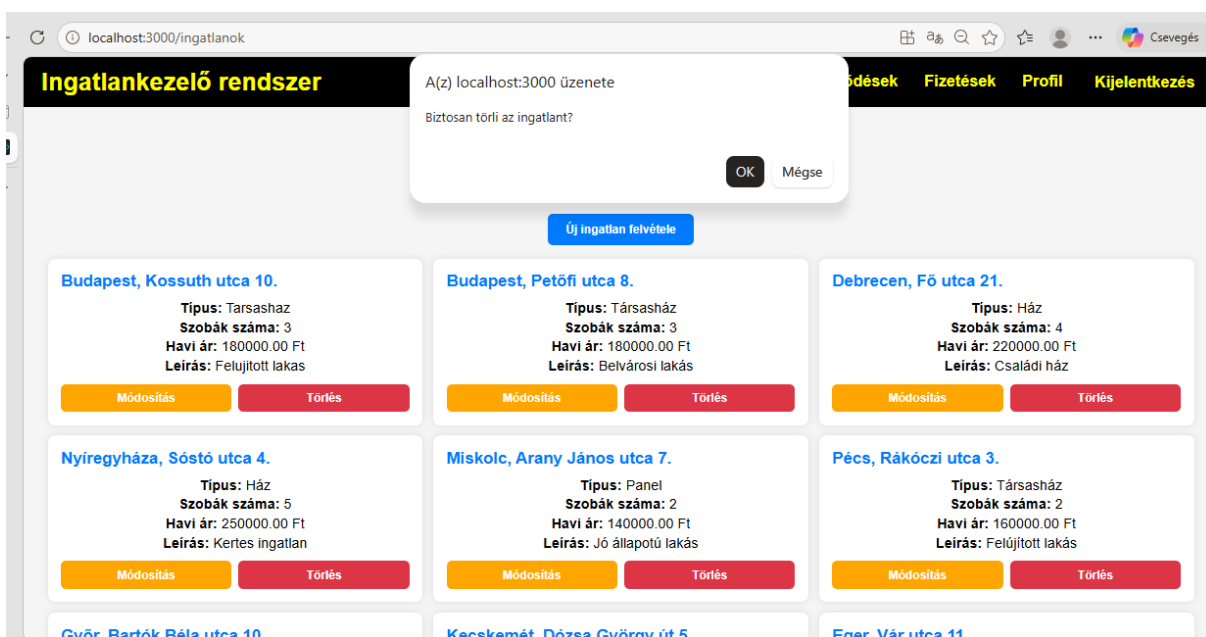
Új ingatlan felvétele

3. Az ingatlan felvétel HTML űrlapra.

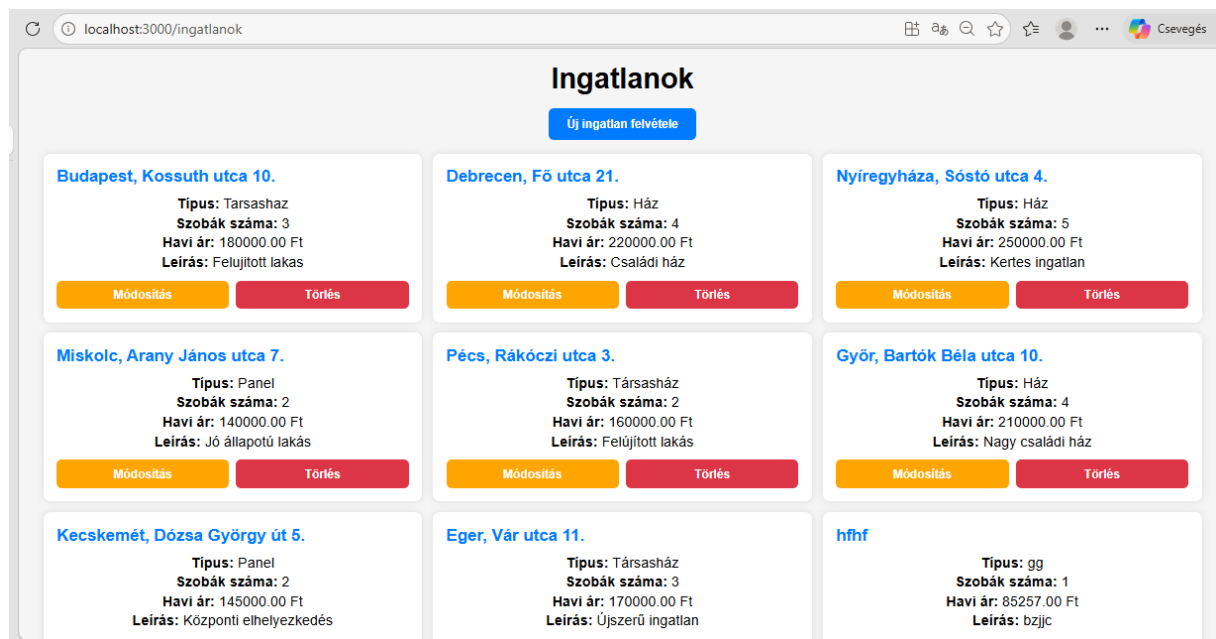
Szakmai Gyakorlat II. – Frontend fejlesztési szakasz
Orosz Kristóf – EYZWG9



4. Részlet az ingatlan módosítás HTML űrlapból.



5. Részlet az ingatlan törlés funkciójáról.



6. Részlet az ingatlan HTML oldalról.

Üzenetek funkciók létrehozása és beállítása

Üzenetek.jsx

Az Uzenetek.jsx komponensben kialakítottam az üzenetkezelő felhasználói felületet. Betöltöm az adatbázisban tárolt üzeneteket, majd azokat kártyás elrendezésben jelenítem meg a felhasználók számára. Lehetővé teszem új üzenetek küldését egy űrlapon keresztül, ahol megadható az üzenet tárgya és tartalma. A kitöltött adatokat a backend API segítségével továbbítom az adatbázis felé. Sikeres küldés után automatikusan frissítem az üzenetek listáját.

```
import { useEffect, useState } from "react";
import "../styles/uzenetek.css";

function Uzenetek() {

  const user =
    JSON.parse(localStorage.getItem("user"));

  const [uzenetek, setUzenetek] = useState([]);

  const [ujUzenet, setUjUzenet] = useState({
    tárgy: "",
    üzenet: ""
  });

  const lekeres = async () => {

    try {

      const valasz = await fetch(
        "http://localhost:3001/api/uzenetlistazas"
```



```
    );

    const adatok = await valasz.json();

    setUzenetek(adatok);

  } catch (err) {

    console.error(err);

  }

};

useEffect(() => {

  lekeres();

}, []);

const kuldes = async (e) => {

  e.preventDefault();

  try {

    const valasz = await fetch(
      "http://localhost:3001/api/uzenetkuldes",
      {
        method: "POST",
        headers: {
          "Content-Type":
            "application/json"
        },
        body: JSON.stringify({
          kuldoid: user.id,
          targy: ujUzenet.targy,
          uzenet: ujUzenet.uzenet
        })
      }
    );

    const adat = await valasz.json();

    console.log(adat);

    alert(
      adat.uzenet ||
```

```
        "Üzenet sikeresen elküldve!"
    );

    setUjUzenet({
        targy: "",
        uzenet: ""
    });

    lekeres();

} catch (err) {

    console.error(err);

    alert("Hiba történt!");

}

};

return (

    <div className="messages-container">

        <h1>Üzenetek</h1>

        <form
            className="message-form"
            onSubmit={kuldes}
        >

            <input
                type="text"
                placeholder="Tárgy"
                value={ujUzenet.targy}
                onChange={e =>
                    setUjUzenet({
                        ...ujUzenet,
                        targy: e.target.value
                    })
                }
                required
            />

            <textarea
                placeholder="Üzenet"
                value={ujUzenet.uzenet}
                onChange={e =>
```

```
        setUjUzenet({
            ...ujUzenet,
            uzenet: e.target.value
        })
    }
    required
/>

<button type="submit">
    Üzenet küldése
</button>

</form>

<div className="messages-grid">

    {uzenetek.map((uzenet) => (

        <div
            className="message-card"
            key={uzenet.id}
        >

            <h3>
                {uzenet.targy}
            </h3>

            <p>
                <strong>
                    Feladó:
                </strong>{" "}
                {uzenet.nev || "Ismeretlen"}
            </p>

            <p>
                <strong>
                    E-mail:
                </strong>{" "}
                {uzenet.email || "-"}
            </p>

            <p>
                <strong>
                    Üzenet:
                </strong>
            </p>

            <div className="message-text">
```

```

        {uzenet.uzenet}
      </div>

      <p className="message-date">

        {
          uzenet.kuldesdatum
            ? new Date(
              uzenet.kuldesdatum
            ).toLocaleString(
              "hu-HU"
            )
            : "-"
        }

      </p>

    </div>

  )))

</div>

</div>

);

}

export default Uzenetek;

```

uzenetek.css

Az uzenetek.css fájlban kialakítottam az üzenetkezelő oldal teljes megjelenését. A stíluslap segítségével középre igazítottam a tartalmat, valamint modern és áttekinthető elrendezést hoztam létre az üzenetküldő űrlap és az üzenetkártyák számára. Az űrlapelemekhez és gombokhoz egységes megjelenést, lekerekített sarkokat és árnyékokat alkalmaztam a felhasználói élmény javítása érdekében. Az üzenetek megjelenítésére rugalmas rácsszerkezetet használtam, amely automatikusan alkalmazkodik a rendelkezésre álló képernyőmérethez.

```

.messages-container {

  max-width: 1200px;
  margin: auto;
  padding: 20px;

}

.messages-container h1 {

```

```
    text-align: center;
    margin-bottom: 25px;
}

.message-form {

    display: flex;
    flex-direction: column;
    gap: 15px;

    background: white;
    padding: 20px;
    border-radius: 12px;

    box-shadow:
        0 2px 10px rgba(0,0,0,0.1);

    margin-bottom: 30px;
}

.message-form input,
.message-form textarea {

    padding: 12px;
    border: 1px solid #ccc;
    border-radius: 8px;
}

.message-form textarea {

    min-height: 120px;
    resize: vertical;
}

.message-form button {

    background: #2563eb;
    color: white;

    border: none;
    padding: 12px;

    border-radius: 8px;
    cursor: pointer;
}
```

```
}

.message-form button:hover {

    background: #1d4ed8;

}

.messages-grid {

    display: grid;

    grid-template-columns:
        repeat(auto-fill,
            minmax(320px, 1fr));

    gap: 20px;

}

.message-card {

    background: white;

    padding: 20px;

    border-radius: 12px;

    box-shadow:
        0 2px 10px rgba(0,0,0,0.1);

}

.message-card h3 {

    margin-bottom: 15px;
    color: #2563eb;

}

.message-text {

    margin-top: 10px;
    margin-bottom: 15px;

    padding: 10px;
```

```
background: #f5f5f5;

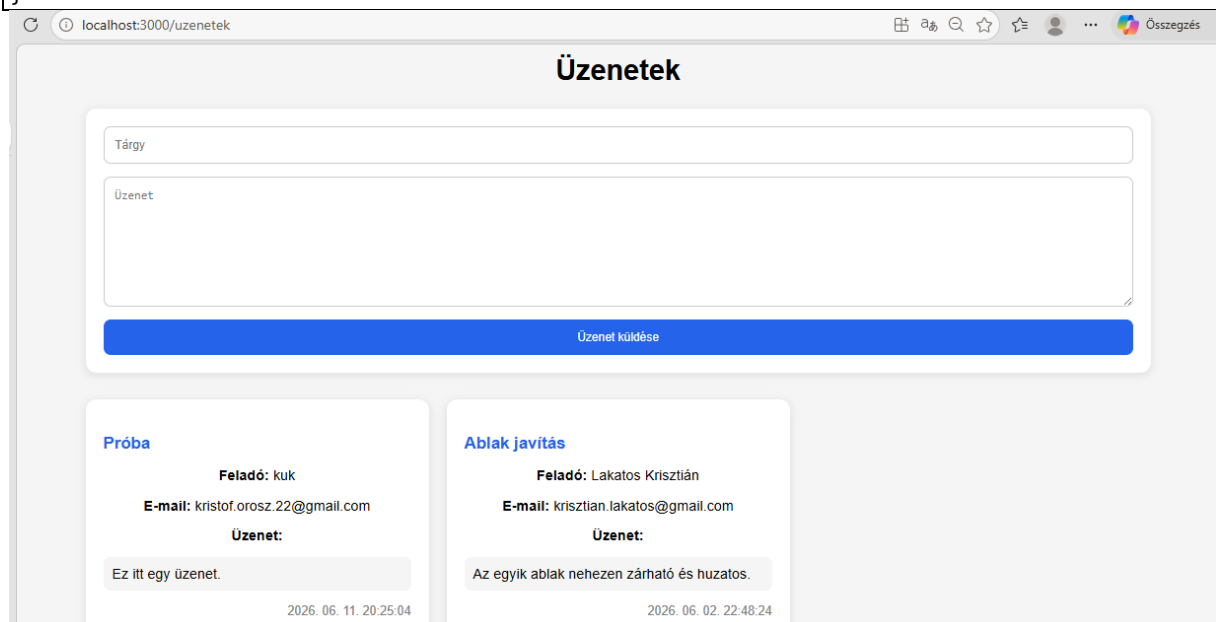
border-radius: 8px;

}

.message-date {

color: gray;
font-size: 14px;
text-align: right;

}
```



1.Kép: Üzenetek HTML oldal megjelenítése és űrlap megjelenítése.

Szerződések funkciók beállítása

Szerződések kezelése

A szerződések kezelése funkció fejlesztése során elkészítettem a szerződések nyilvántartására szolgáló felhasználói felületet. Az űrlapon lehetővé tettem új szerződések létrehozását, ahol kiválasztható az ingatlan és a bérlő, valamint megadhatók a szerződéshez tartozó adatok. A rendszer automatikusan betölti az adatbázisban található ingatlanokat és bérlőket, így a felhasználónak nem kell manuálisan megadnia az azonosítókat. Megvalósítottam a szerződések módosításának lehetőségét is, amely során a kiválasztott szerződés adatai automatikusan betöltődnek az űrlapba szerkesztés céljából. A funkció tesztelése során ellenőriztem az új szerződések létrehozását, a meglévő adatok módosítását, valamint a szerződések törlését is. A rendszerben kialakítottam a szerződések státusz szerinti szűrésének lehetőségét. A felhasználó kiválaszthatja, hogy az összes, az aktív vagy a lejárt szerződéseket szeretné megjeleníteni. A kiválasztott szűrő alapján a frontend dinamikusan szűri a megjelenített adatokat. Ezzel jelentősen egyszerűbbé vált a nagyobb mennyiségű szerződés áttekintése. A funkció működését különböző státuszú tesztadatokkal ellenőriztem.

```
import { useEffect, useState } from "react";
```

```
import "../styles/szerzodesek.css";

function Szerzodesek() {

  const [szerzodesek, setSzerzodesek] = useState([]);
  const [berlok, setBerlok] = useState([]);
  const [ingatlanok, setIngatlanok] = useState([]);
  const [szerkesztettId, setSzerkesztettId] = useState(null);
  const [statuszSzuro, setStatuszSzuro] = useState("Mind");

  const [ujSzerzodes, setUjSzerzodes] = useState({
    ingatlanid: "",
    berloid: "",
    kezdetdatum: "",
    vegdatum: "",
    havidij: "",
    statusz: "Aktív"
  });

  const szerzodesekBetoltese = async () => {

    try {

      const valasz = await fetch(
        "http://localhost:3001/api/szerzodeslistazas"
      );

      const adatok = await valasz.json();

      setSzerzodesek(adatok);

    } catch (err) {

      console.error(err);

    }

  };

  const berlokBetoltese = async () => {

    try {

      const valasz = await fetch(
        "http://localhost:3001/api/berlolistazas"
      );

      const adatok = await valasz.json();
```



```
        setBerlok(adatok);

    } catch (err) {

        console.error(err);

    }

};

const ingatlanokBetoltese = async () => {

    try {

        const valasz = await fetch(
            "http://localhost:3001/api/ingatlanlistazas"
        );

        const adatok = await valasz.json();

        setIngatlanok(adatok);

    } catch (err) {

        console.error(err);

    }

};

useEffect(() => {

    szerzodesekBetoltese();
    berlokBetoltese();
    ingatlanokBetoltese();

}, []);

const mentes = async (e) => {

    e.preventDefault();

    try {

        let url =
            "http://localhost:3001/api/szerzodesletrehozas";
```

```
let method = "POST";

if (szerkesztettId) {

    url =
        `http://localhost:3001/api/szerzodesmodositas/${szerkesztettId}`;

    method = "PUT";

}

const valasz = await fetch(
    url,
    {
        method,
        headers: {
            "Content-Type":
                "application/json"
        },
        body: JSON.stringify(
            ujSzerzodes
        )
    }
);

const adat =
    await valasz.json();

alert(adat.uzenet);

setUjSzerzodes({
    ingatlanid: "",
    berloid: "",
    kezdetdatum: "",
    vegdatum: "",
    havidij: "",
    statusz: "Aktív"
});

setSzerkesztettId(null);

szerzodesekBetoltese();

} catch (err) {

    console.error(err);

    alert("Hiba történt!");
```

```
    }  
};  
  
const torles = async (id) => {  
  
    if (  
        !window.confirm(  
            "Biztosan töröld a szerződést?"  
        )  
    ) {  
        return;  
    }  
  
    try {  
  
        const valasz = await fetch(  
            `http://localhost:3001/api/szerzodestorles/${id}`,  
            {  
                method: "DELETE"  
            }  
        );  
  
        const adat =  
            await valasz.json();  
  
        alert(adat.uzenet);  
  
        szerzodesekBetoltese();  
  
    } catch (err) {  
  
        console.error(err);  
  
    }  
};  
  
return (  
  
    <div className="tenant-container">  
  
        <h1>Szerződések</h1>  
  
        <form  
            className="tenant-form"  
            onSubmit={mentes}  

```

```
>

<select
  value={
    ujSzerzodes.ingatlanid
  }
  onChange={(e) =>
    setUjSzerzodes({
      ...ujSzerzodes,
      ingatlanid:
        e.target.value
    })
  }
  required
>

  <option value="">
    Válassz ingatlant
  </option>

  {ingatlanok.map(
    (ingatlan) => (

      <option
        key={
          ingatlan.id
        }
        value={
          ingatlan.id
        }
      >
        {
          ingatlan.cim
        }
      </option>

    )
  )}

</select>

<select
  value={
    ujSzerzodes.berloid
  }
  onChange={(e) =>
    setUjSzerzodes({
      ...ujSzerzodes,
```

```
        berloid:
          e.target.value
      })
    }
    required
  >

  <option value="">
    Válassz bérlőt
  </option>

  {berlok.map(
    (berlo) => (

      <option
        key={
          berlo.id
        }
        value={
          berlo.id
        }
      >
        {
          berlo.nev
        }
      </option>

    )
  )}

</select>

<input
  type="date"
  value={
    ujSzerzodes.kezdetdatum
  }
  onChange={(e) =>
    setUjSzerzodes({
      ...ujSzerzodes,
      kezdetdatum:
        e.target.value
    })
  }
  required
/>

<input
```

```
        type="date"
        value={
            ujSzerzodes.vegdatum
        }
        onChange={(e) =>
            setUjSzerzodes({
                ...ujSzerzodes,
                vegdatum:
                    e.target.value
            })
        }
        required
    />

    <input
        type="number"
        placeholder="Havi díj"
        value={
            ujSzerzodes.havidij
        }
        onChange={(e) =>
            setUjSzerzodes({
                ...ujSzerzodes,
                havidij:
                    e.target.value
            })
        }
        required
    />

    <select
        value={
            ujSzerzodes.statusz
        }
        onChange={(e) =>
            setUjSzerzodes({
                ...ujSzerzodes,
                statusz:
                    e.target.value
            })
        }
    >

        <option value="Aktív">
            Aktív
        </option>

        <option value="Lejárt">
```

```
        Lejárt
      </option>

    </select>

    <button
      type="submit"
    >
      {
        szerkesztettId
          ? "Módosítás mentése"
          : "Szerződés létrehozása"
      }
    </button>

  </form>

  <div className="filter-container">

    <select
      value={statuszSzuro}
      onChange={(e) =>
        setStatuszSzuro(
          e.target.value
        )
      }
    >

      <option value="Mind">
        Összes szerződés
      </option>

      <option value="Aktív">
        Aktív
      </option>

      <option value="Lejárt">
        Lejárt
      </option>

    </select>

  </div>

  <div className="tenant-grid">

    {szerzodesek
      .filter((szerzodes) => {
```

```
        if (
            statuszSzuro ===
            "Mind"
        ) {
            return true;
        }

        return (
            szerzodes.statusz ===
            statuszSzuro
        );
    })
    .map(
        (szerzodes) => (

            <div
                className="tenant-card"
                key={
                    szerzodes.id
                }
            >

                <h3>
                    {
                        szerzodes.ingatlancim
                    }
                </h3>

                <p>
                    <strong>
                        Bérlő:
                    </strong>{" "}
                    {
                        szerzodes.berlonev
                    }
                </p>

                <p>
                    <strong>
                        Kezdet:
                    </strong>{" "}
                    {
                        new Date(
                            szerzodes.kezdetdatum
                        ).toLocaleDateString(
                            "hu-HU"
                        )
                    }
                </p>
            </div>
        )
    )
}
```



```
        )
    }
</p>

<p>
    <strong>
        Vége:
    </strong>{" "}
    {
        new Date(
            szerzodes.vegdatum
        ).toLocaleDateString(
            "hu-HU"
        )
    }
</p>

<p>
    <strong>
        Havi díj:
    </strong>{" "}
    {
        szerzodes.havidij
    } Ft
</p>

<p>

    <strong>
        Státusz:
    </strong>

    <span
        className={
            szerzodes.statusz ===
            "Aktív"
            ? "active-status"
            : "expired-status"
        }
    >
        {" "}
        {
            szerzodes.statusz
        }
    </span>

</p>
```

```
<div className="buttons">

  <button
    type="button"
    className="edit-btn"
    onClick={() => {

      setSzerkesztettId(
        szerzodes.id
      );

      setUjSzerzodes({
        ingatlanid:
          szerzodes.ingatlanid,
        berloid:
          szerzodes.berloid,
        kezdetdatum:
          szerzodes.kezdetdatum?.split("T")[0] ||
          szerzodes.kezdetdatum,
        vegdatum:
          szerzodes.vegdatum?.split("T")[0] ||
          szerzodes.vegdatum,
        havidij:
          szerzodes.havidij,
        statusz:
          szerzodes.statusz
      });

      window.scrollTo({
        top: 0,
        behavior: "smooth"
      });

    }}
  >
    Módosítás
  </button>

  <button
    type="button"
    className="delete-btn"
    onClick={() =>
      torles(
        szerzodes.id
      )
    }
  >
    Törlés
```

```
                </button>

            </div>

        </div>

    )
    })

</div>

</div>

);

}

export default Szerzodesek;
```

Szerződések oldal stíluslapja

A szerződések oldal megjelenítéséhez külön CSS stíluslapot készítettem. A célom egy letisztult, könnyen áttekinthető és felhasználóbarát felület kialakítása volt. A szerződések kártyás elrendezésben jelennek meg, ami megkönnyíti az adatok áttekintését. Külön stílusokat alkalmaztam az űrlapokra, a gombokra és a státuszjelölésekre is. A reszponzív kialakításnak köszönhetően az oldal kisebb képernyőkön és mobil eszközökön is megfelelően használható.

```
.tenant-container {
    max-width: 1400px;
    margin: 30px auto;
    padding: 20px;
}

.tenant-container h1 {
    text-align: center;
    margin-bottom: 30px;
    font-size: 42px;
}

.tenant-form {
    display: flex;
    flex-wrap: wrap;
    gap: 15px;
    background: white;
    padding: 20px;
    border-radius: 12px;
    box-shadow: 0 3px 10px rgba(0,0,0,0.15);
    margin-bottom: 30px;
```

```
}

.tenant-form select,
.tenant-form input {
  flex: 1;
  min-width: 180px;
  padding: 12px;
  border: 1px solid #ccc;
  border-radius: 8px;
}

.tenant-form button {
  background: #2e7d32;
  color: white;
  border: none;
  padding: 12px 25px;
  border-radius: 8px;
  cursor: pointer;
}

.tenant-form button:hover {
  background: #1b5e20;
}

.tenant-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill,minmax(320px,1fr));
  gap: 20px;
}

.tenant-card {
  background: white;
  padding: 20px;
  border-radius: 12px;
  box-shadow: 0 3px 10px rgba(0,0,0,0.15);
}

.tenant-card h3 {
  margin-bottom: 15px;
  color: #2e7d32;
}

.tenant-card p {
  margin: 8px 0;
}

.buttons {
  margin-top: 15px;
```

```
    display: flex;
    gap: 10px;
}

.edit-btn {
    background: #1976d2;
    color: white;
    border: none;
    padding: 10px 15px;
    border-radius: 8px;
    cursor: pointer;
}

.edit-btn:hover {
    background: #0d47a1;
}

.delete-btn {
    background: #d32f2f;
    color: white;
    border: none;
    padding: 10px 15px;
    border-radius: 8px;
    cursor: pointer;
}

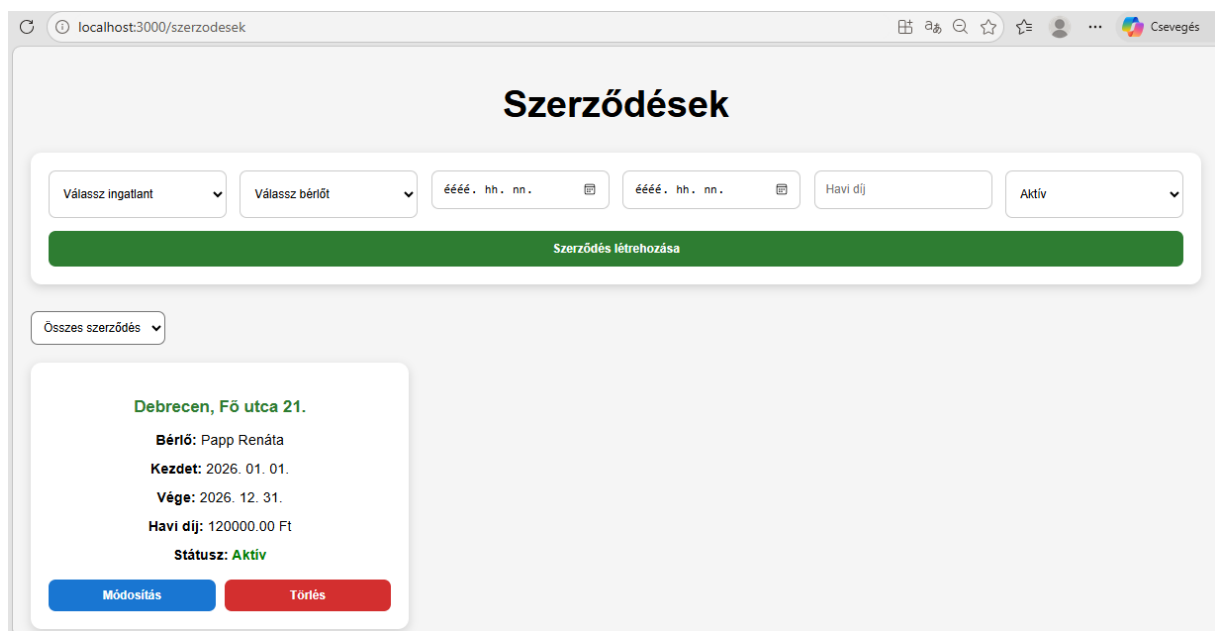
.delete-btn:hover {
    background: #b71c1c;
}

.filter-container {
    margin: 20px 0;
}

.filter-container select {
    padding: 10px;
    border-radius: 8px;
}

.active-status {
    color: green;
    font-weight: bold;
}

.expired-status {
    color: red;
    font-weight: bold;
}
```



1.Kép: Szerződések HTML oldal kimenete.

Fizetések létrehozása

1. Fizetesek.jsx

A fizetések kezeléséhez elkészítettem a Fizetesek.jsx komponenst. A komponensben React state-ek segítségével tároltam a fizetések, a szerződések és az űrlap adatait. Gondoskodtam arról, hogy az oldal betöltésekor automatikusan lekérdeződjenek a szükséges adatok a backendből. Az űrlap segítségével lehetővé tettem új fizetések rögzítését és a meglévő fizetések módosítását.

```
import { useEffect, useState } from "react";
import "../styles/fizetesek.css";

function Fizetesek() {

  const [fizetesek, setFizetesek] = useState([]);
  const [szerzodesek, setSzerzodesek] = useState([]);
  const [szerkesztettId, setSzerkesztettId] = useState(null);
  const [statuszSzuro, setStatuszSzuro] = useState("Mind");

  const [ujFizetes, setUjFizetes] = useState({
    szerzodesid: "",
    osszeg: "",
    fizetesdatum: "",
    statusz: "Teljesítve",
    fizetesmod: "",
    megjegyzes: ""
  });
}
```

```
const fizetesekBetoltese = async () => {

  try {

    const valasz = await fetch(
      "http://localhost:3001/api/fizeteslistazas"
    );

    const adatok = await valasz.json();

    setFizetesek(adatok);

  } catch (err) {

    console.error(err);

  }

};

const szerzodesekBetoltese = async () => {

  try {

    const valasz = await fetch(
      "http://localhost:3001/api/szerzodeslistazas"
    );

    const adatok = await valasz.json();

    setSzerzodesek(adatok);

  } catch (err) {

    console.error(err);

  }

};

useEffect(() => {

  fizetesekBetoltese();
  szerzodesekBetoltese();

}, []);

const mentes = async (e) => {
```

```
e.preventDefault();

try {

    let url =
        "http://localhost:3001/api/fizetesletrehozas";

    let method = "POST";

    if (szerkesztettId) {

        url =
            `http://localhost:3001/api/fizetesmodositas/${szerkesztettId}`;

        method = "PUT";

    }

    const valasz = await fetch(
        url,
        {
            method,
            headers: {
                "Content-Type":
                    "application/json"
            },
            body: JSON.stringify(
                ujFizetes
            )
        }
    );

    const adat =
        await valasz.json();

    alert(adat.uzenet);

    setUjFizetes({
        szerzodesid: "",
        osszeg: "",
        fizetesdatum: "",
        statusz: "Teljesítve",
        fizetesmod: "",
        megjegyzes: ""
    });

    setSzerkesztettId(null);
```



```
        fizetesekBetoltese();

    } catch (err) {

        console.error(err);

        alert("Hiba történt!");

    }

};

const torles = async (id) => {

    if (
        !window.confirm(
            "Biztosan töröld a fizetést?"
        )
    ) {
        return;
    }

    try {

        const valasz = await fetch(
            `http://localhost:3001/api/fizetestorles/${id}`,
            {
                method: "DELETE"
            }
        );

        const adat =
            await valasz.json();

        alert(adat.uzenet);

        fizetesekBetoltese();

    } catch (err) {

        console.error(err);

    }

};

return (
```

```
<div className="tenant-container">

  <h1>Fizetések</h1>

  <form
    className="tenant-form"
    onSubmit={mentes}
  >

    <select
      value={
        ujFizetes.szerzodesid
      }
      onChange={(e) =>
        setUjFizetes({
          ...ujFizetes,
          szerzodesid:
            e.target.value
        })
      }
      required
    >

      <option value="">
        Válassz szerződést
      </option>

      {szerzodesek.map(
        (szerzodes) => (

          <option
            key={
              szerzodes.id
            }
            value={
              szerzodes.id
            }
          >
            {
              szerzodes.berlonev
            }
          </option>

        )
      )}

    </select>
```

```
<input
  type="number"
  placeholder="Összeg"
  value={
    ujFizetes.osszeg
  }
  onChange={(e) =>
    setUjFizetes({
      ...ujFizetes,
      osszeg:
        e.target.value
    })
  }
  required
/>

<input
  type="date"
  value={
    ujFizetes.fizetesdatum
  }
  onChange={(e) =>
    setUjFizetes({
      ...ujFizetes,
      fizetesdatum:
        e.target.value
    })
  }
  required
/>

<select
  value={
    ujFizetes.statusz
  }
  onChange={(e) =>
    setUjFizetes({
      ...ujFizetes,
      statusz:
        e.target.value
    })
  }
>

  <option value="Teljesítve">
    Teljesítve
  </option>
```

```
        <option value="Függőben">
            Függőben
        </option>

        <option value="Sikertelen">
            Sikertelen
        </option>

    </select>

    <input
        type="text"
        placeholder="Fizetési mód"
        value={
            ujFizetes.fizetesmod
        }
        onChange={(e) =>
            setUjFizetes({
                ...ujFizetes,
                fizetesmod:
                    e.target.value
            })
        }
        required
    />

    <input
        type="text"
        placeholder="Megjegyzés"
        value={
            ujFizetes.megjegyzes
        }
        onChange={(e) =>
            setUjFizetes({
                ...ujFizetes,
                megjegyzes:
                    e.target.value
            })
        }
    />

    <button
        type="submit"
    >
        {
            szerkesztettId
                ? "Módosítás mentése"
```

```
        : "Fizetés rögzítése"
      }
    </button>

  </form>

  <div className="filter-container">

    <select
      value={statuszSzuro}
      onChange={(e) =>
        setStatuszSzuro(
          e.target.value
        )
      }
    >

      <option value="Mind">
        Összes fizetés
      </option>

      <option value="Teljesítve">
        Teljesítve
      </option>

      <option value="Függőben">
        Függőben
      </option>

      <option value="Sikertelen">
        Sikertelen
      </option>

    </select>

  </div>

  <div className="tenant-grid">

    {fizetesek
      .filter((fizetes) => {

        if (
          statuszSzuro ===
            "Mind"
        ) {
          return true;
        }
      })
    }
```

```
        return (
            fizetes.statusz ===
            statuszSzuro
        );
    })
    .map(
        (fizetes) => (
            <div
                className="tenant-card"
                key={
                    fizetes.id
                }
            >

                <h3>
                    {
                        fizetes.berlonev
                    }
                </h3>

                <p>
                    <strong>
                        Összeg:
                    </strong>{" "}
                    {
                        fizetes.osszeg
                    } Ft
                </p>

                <p>
                    <strong>
                        Dátum:
                    </strong>{" "}
                    {
                        new Date(
                            fizetes.fizetesdatum
                        ).toLocaleDateString(
                            "hu-HU"
                        )
                    }
                </p>

                <p>
                    <strong>
                        Fizetési mód:
```

```

        </strong>{" "}
        {
            fizetes.fizetesmod
        }
    </p>

    {
        fizetes.megjegyzes &&
        (
            <p>
                <strong>
                    Megjegyzés:
                </strong>{" "}
                {
                    fizetes.megjegyzes
                }
            </p>
        )
    }

    <p>

        <strong>
            Státusz:
        </strong>

        <span
            className={
                fizetes.statusz === "Teljesítve"
                ? "active-status"
                : fizetes.statusz === "Függőben"
                ? "pending-status"
                : "expired-status"
            }
        >
            {" "}
            {
                fizetes.statusz
            }
        </span>

    </p>

    <div className="buttons">

        <button
            type="button"
            className="edit-btn"

```

```

onClick={() => {

    setSzerkesztettId(
        fizetes.id
    );

    setUjFizetes({
        szerzodesid:
            fizetes.szerzodesid,
        osszeg:
            fizetes.osszeg,
        fizetesdatum:
            fizetes.fizetesdatum?.split("T")[0] ||
            fizetes.fizetesdatum,
        statusz:
            fizetes.statusz,
        fizetesmod:
            fizetes.fizetesmod,
        megjegyzes:
            fizetes.megjegyzes || ""
    });

    window.scrollTo({
        top: 0,
        behavior: "smooth"
    });

}}
>
    Módosítás
</button>

<button
    type="button"
    className="delete-btn"
    onClick={() =>
        torles(
            fizetes.id
        )
    }
>
    Törlés
</button>

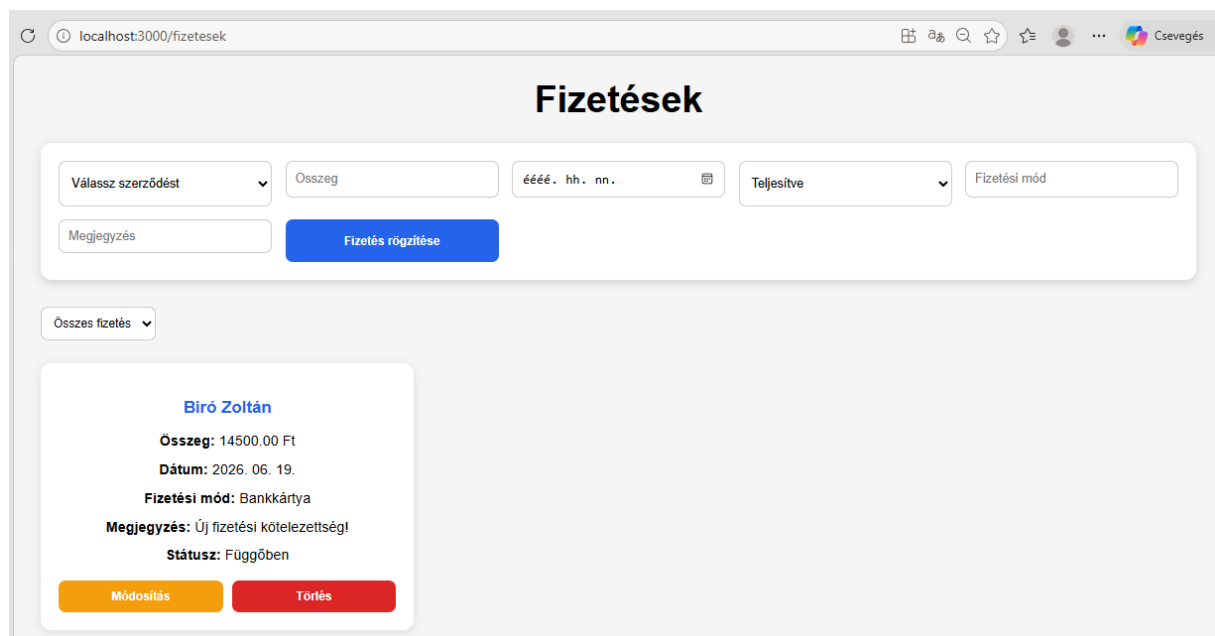
</div>

</div>

```



```
    )  
  })  
  
  </div>  
  
</div>  
  
);  
  
}  
  
export default Fizetesek;
```



1.Kép: Fizetések HTML oldala a funkciókkal ellátva.

Bérlők oldalak és funkciók beállítása

Berlok.jsx

A Berlok.jsx fájlban elkészítettem a bérlők adatainak megjelenítésére szolgáló React komponenst. Az oldal betöltésekor lekérem a backendből a bérlők adatait, majd eltárolom azokat a komponens állapotában. A kapott adatokat kártyás elrendezésben jelenítem meg, így a felhasználók könnyen áttekinthetik a bérlők adatait. Minden kártyán megjelenítem a bérlő nevét, e-mail címét, telefonszámát, személyi számát és lakcímét.

```
import { useEffect, useState } from "react";  
import "../styles/berlok.css";  
  
function Berlok() {
```

```
const [berlok, setBerlok] = useState([]);

const berlokBetoltese = async () => {

  try {

    const response = await fetch(
      "http://localhost:3001/api/berlolistazas"
    );

    const data = await response.json();

    setBerlok(
      Array.isArray(data)
        ? data
        : []
    );

  } catch (error) {

    console.error(error);

    alert(
      "Nem sikerült betölteni a bérlőket!"
    );

  }
};

useEffect(() => {

  berlokBetoltese();

}, []);

return (

  <div className="tenant-container">

    <h1>Bérlők</h1>

    <div className="tenant-grid">

      {berlok.map((berlo) => (

        <div
          key={berlo.id}
```

```
        className="tenant-card"
    >

        <h3>
            {berlo.nev}
        </h3>

        <p>
            <strong>
                E-mail:
            </strong>{" "}
            {
                berlo.email ||
                "Nincs megadva"
            }
        </p>

        <p>
            <strong>
                Telefonszám:
            </strong>{" "}
            {
                berlo.telefonszam ||
                "Nincs megadva"
            }
        </p>

        <p>
            <strong>
                Személyi szám:
            </strong>{" "}
            {
                berlo.szemelyiszam ||
                "Nincs megadva"
            }
        </p>

        <p>
            <strong>
                Lakcím:
            </strong>{" "}
            {
                berlo.lakcim ||
                "Nincs megadva"
            }
        </p>

    </div>
```

```
        )})  
  
    </div>  
  
</div>  
  
);  
  
}  
  
export default Berlok;
```

berlok.css

A berlok.css fájlban kialakítottam a bérlok oldalának megjelenését. A stíluslap segítségével középre igazítottam a tartalmat, valamint modern és áttekinthető kártyás elrendezést hoztam létre. Árnyékokat, lekerekített sarkokat és megfelelő térközöket alkalmaztam a jobb felhasználói élmény érdekében. A rácsszerkezet automatikusan alkalmazkodik a képernyő méretéhez, így az oldal reszponzív módon jelenik meg.

```
.tenant-container {  
    max-width: 1400px;  
    margin: 0 auto;  
    padding: 20px;  
}  
  
.tenant-container h1 {  
    text-align: center;  
    margin-bottom: 20px;  
}  
  
.add-btn {  
    display: block;  
    margin: 0 auto 20px auto;  
    padding: 12px 20px;  
    background-color: #007bff;  
    color: white;  
    border: none;  
    border-radius: 8px;  
    cursor: pointer;  
    font-weight: bold;  
}  
  
.add-btn:hover {  
    background-color: #0056b3;  
}
```

```
.tenant-form {
  background: white;
  padding: 20px;
  border-radius: 10px;
  margin-bottom: 25px;
  box-shadow: 0 0 15px rgba(0,0,0,0.15);
}

.tenant-form input,
.tenant-form textarea {
  width: 100%;
  padding: 12px;
  margin-bottom: 10px;
  border: 1px solid #ccc;
  border-radius: 6px;
  box-sizing: border-box;
}

.tenant-form button {
  width: 100%;
  padding: 12px;
  background: #28a745;
  color: white;
  border: none;
  border-radius: 6px;
  font-weight: bold;
  cursor: pointer;
}

.tenant-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(380px, 1fr));
  gap: 20px;
}

.tenant-card {
  background: white;
  padding: 20px;
  border-radius: 12px;
  box-shadow: 0 0 15px rgba(0,0,0,0.12);
  transition: 0.3s;
}

.tenant-card:hover {
  transform: translateY(-5px);
}

.tenant-card h3 {
```

```
    text-align: center;
    color: #007bff;
    margin-bottom: 15px;
}

.tenant-card p {
    margin: 8px 0;
    line-height: 1.5;
}

.buttons {
    display: flex;
    gap: 10px;
    margin-top: 15px;
}

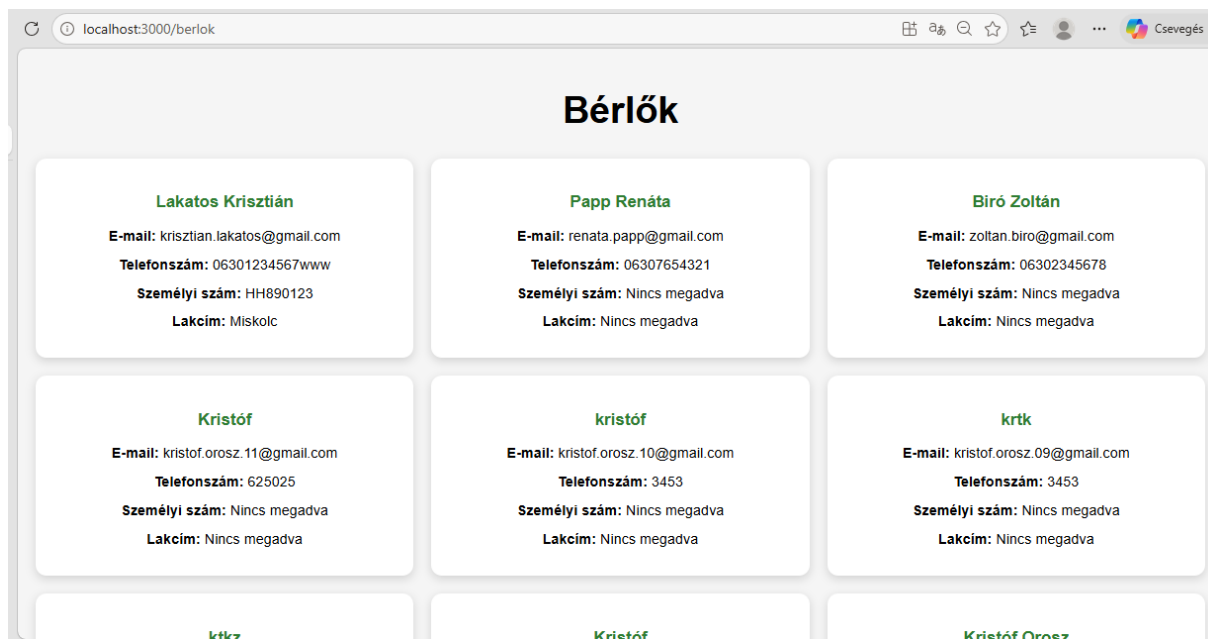
.edit-btn {
    flex: 1;
    background: orange;
    color: white;
    border: none;
    padding: 10px;
    border-radius: 6px;
    font-weight: bold;
    cursor: pointer;
}

.delete-btn {
    flex: 1;
    background: #dc3545;
    color: white;
    border: none;
    padding: 10px;
    border-radius: 6px;
    font-weight: bold;
    cursor: pointer;
}

.assign-btn {
    flex: 1;
    background: #28a745;
    color: white;
    border: none;
    padding: 10px;
    border-radius: 6px;
    font-weight: bold;
    cursor: pointer;
}
```

Szakmai Gyakorlat II. – Frontend fejlesztési szakasz

Orosz Kristóf – EYZWG9



1.Kép: Bérlők HTML oldala.